

Universidad Libre De Colombia

Facultad De Ingeniería

Manual Técnico

Proyecto 4: Sistema de Gestión de Mascotas para una Clínica Veterinaria

Arquitectura, diseño y operación del sistema

Presenta:

Santiago Enrique Arquez Cervantes, Cristian David Diaz Alarcón, Ginneth Natalia González Puentes, Doncel Castro Stephania

Tutor:

Miguel Hernández Bejarano

Bogotá D.C.

1. Descripción general

El sistema sirve para llevar el control de propietarios, mascotas y citas veterinarias. Está hecho con tres microservicios independientes que ofrecen APIs REST y se comunican entre sí por HTTP. Cada uno tiene su propia base de datos MySQL y corre en su propio contenedor Docker. Este manual explica cómo está armado el sistema: la arquitectura, el modelo de datos, las APIs, las reglas de negocio, la comunicación entre servicios, la configuración, el despliegue y las pruebas. Está pensado para quien vaya a mantenerlo o ampliarlo. El código fuente está en el repositorio https://github.com/SantiagoArquez/Veterianrias.

2. Tecnologías utilizadas

Tecnología	Versión	Uso
Java	21 (LTS)	Lenguaje de programación (se compila con release 21).
Spring Boot	3.3.5	Framework base de los microservicios (Web, Validation y Data JPA).
Hibernate ORM	6.5.3	Implementación de JPA: conecta los objetos con las tablas.
MySQL	8.4	Base de datos de cada servicio (corre en un contenedor).
springdoc-openapi	2.6.0	Documentación OpenAPI y Swagger UI.
JUnit 5 y Mockito	Incluidas en Spring Boot 3.3.5	Pruebas unitarias y de integración.
H2	Incluida en Spring Boot 3.3.5	Base de datos en memoria, solo para las pruebas.
Maven	3.9	Manejo de dependencias y compilación.
Docker y Docker Compose	v2	Contenedores y orquestación en local.

3. Arquitectura

3.1 Visión general

Usamos el patrón de una base de datos por servicio: cada microservicio es dueño de sus datos y ningún otro servicio entra directamente a su base. Por eso no hay claves foráneas

entre servicios. Pet.ownerId y Appointment.petId son referencias lógicas, y para saber si son válidas se le pregunta por REST al servicio que guarda ese dato.

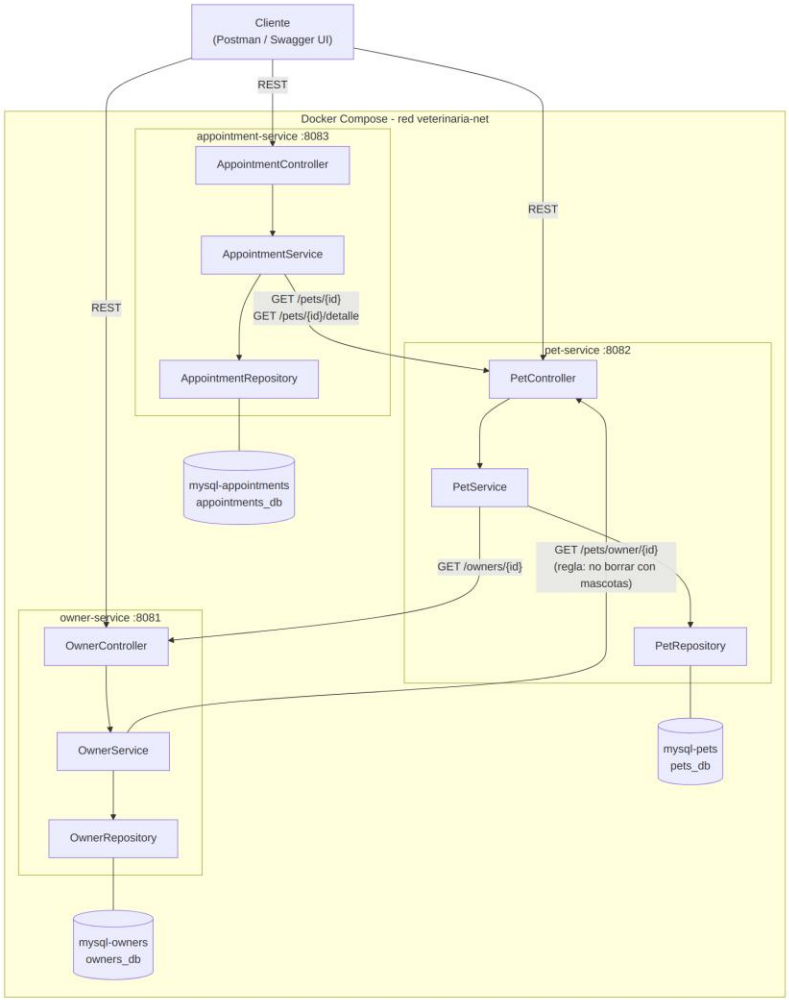


Figura 1. Diagrama de arquitectura.

3.2 Servicios y puertos

Servicio	Puerto	Base de datos	Responsabilidad
Owner-Service	8081	owners_db	Maneja los propietarios. Le pregunta a Pet-Service para no dejar borrar propietarios que tengan mascotas.
Pet-Service	8082	pets_db	Maneja las mascotas. Le pregunta a Owner-Service para validar al propietario y armar el detalle.

Servicio	Puerto	Base de datos	Responsabilidad
Appointment-Service	8083	appointments_db	Maneja las citas. Le pregunta a Pet-Service para validar la mascota y armar la cita completa.

3.3 Arquitectura interna de cada servicio

Todos los servicios están organizados de la misma forma, por capas, con un archivo para cada una dentro del paquete base com.clinica. servicio como se pueden dar cuenta

Archivo	Función
...ServiceApplication.java	Clase principal. Arranca Spring Boot y crea el bean RestTemplate con sus tiempos máximos de conexión y lectura.
Owner.java, Pet.java, Appointment.java	Entidad JPA ligada a su tabla. Trae los enumerados del dominio (Sexo, Estado).
...Repository.java	Interfaz de Spring Data JPA: las consultas salen del nombre del método (existsByDocumento, findByOwnerId, etc.).
Dtos.java	Records de entrada y salida de la API, con sus validaciones de Bean Validation, y el formato de error ApiError.
Errores.java	Excepciones de negocio (NoEncontrado, ReglaNegocio, Conflicto, ServicioNoDisponible) y el manejador global que las convierte en respuestas HTTP.
PetClient.java / OwnerClient.java	Cliente HTTP para llamar a otro microservicio; convierte sus fallos en excepciones propias.
...Service.java	Capa de servicio: aquí están las reglas de negocio y las transacciones.
...Controller.java	Controlador REST: recibe la petición, la valida con @Valid y se la pasa al servicio.
application.yml	Configuración del servicio (puerto, base de datos, URLs de otros servicios y timeouts).

3.4 Modelo de clases

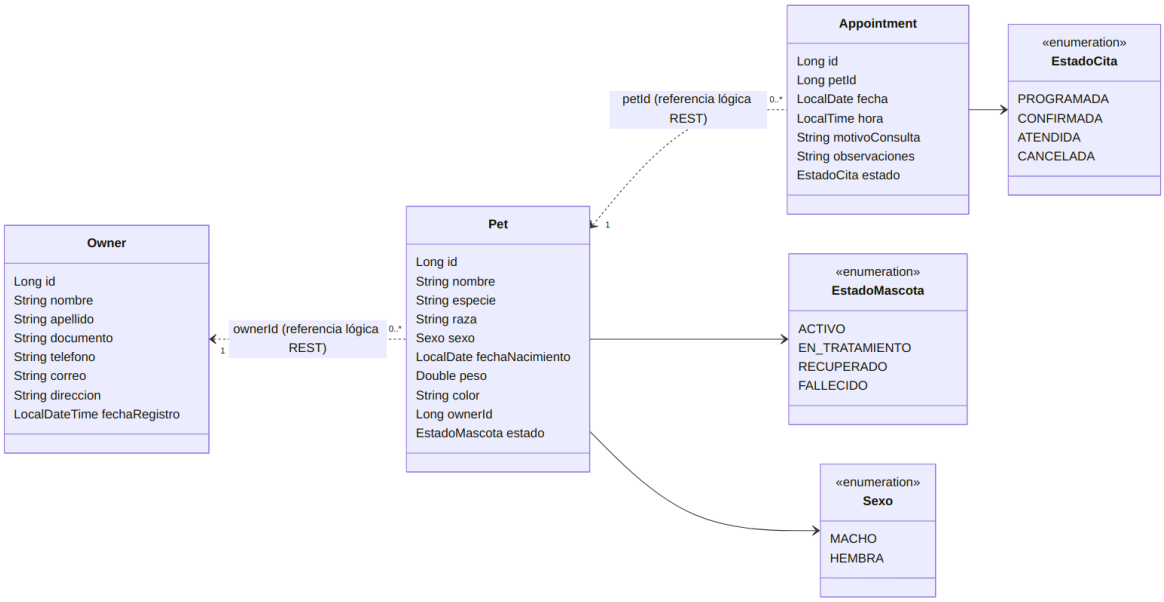


Figura 2. Diagrama de clases.

4. Modelo de datos

Las tablas se crean y se actualizan solas cuando arranca cada servicio , Los enumerados se guardan como texto.

Tabla owners (owners_db)

Columna	Tipo	Restricciones
id	BIGINT	Clave primaria, autoincremental.
nombre	VARCHAR(80)	Obligatorio.
apellido	VARCHAR(80)	Obligatorio.
documento	VARCHAR(30)	Obligatorio, único (uk_owner_documento).
telefono	VARCHAR(30)	Obligatorio.
correo	VARCHAR(120)	Obligatorio, único (uk_owner_correo).
direccion	VARCHAR(200)	Opcional.
fecha_registro	DATETIME	Obligatorio; se llena sola al crear el registro y después no cambia.

Tabla pets (pets_db)

Columna	Tipo	Restricciones
id	BIGINT	Clave primaria, autoincremental.
nombre	VARCHAR(80)	Obligatorio.
especie	VARCHAR(60)	Obligatorio.
raza	VARCHAR(80)	Obligatorio.
sexo	VARCHAR(10)	Obligatorio: MACHO o HEMBRA.
fecha_nacimiento	DATE	Obligatorio; no puede ser una fecha futura.
peso	DOUBLE	Obligatorio; mayor que cero.
color	VARCHAR(60)	Opcional.
owner_id	BIGINT	Obligatorio; referencia lógica a owners.id; con índice idx_pet_owner.
estado	VARCHAR(20)	Obligatorio: ACTIVO, EN_TRATAMIENTO, RECUPERADO o FALLECIDO.

Tabla appointments (appointments_db)

Columna	Tipo	Restricciones
id	BIGINT	Clave primaria, autoincremental.
pet_id	BIGINT	Obligatorio; referencia lógica a pets.id; con índice idx_appointment_pet.
fecha	DATE	Obligatorio; índice compuesto con hora.
hora	TIME	Obligatorio.
motivo_consulta	VARCHAR(255)	Obligatorio.
observaciones	VARCHAR(1000)	Opcional.
estado	VARCHAR(20)	Obligatorio: PROGRAMADA, CONFIRMADA, ATENDIDA o CANCELADA.

5. APIs REST

todas las apis trabajan con json. las fechas se mandan con el formato yyyy-MM-dd y las horas con HH:mm. cada servicio tiene su documentación interactiva en la ruta /swagger-ui.html, y la encuentras en el puerto de ese mismo servicio.

5.1 Owner-Service (puerto 8081)

Método	Ruta	Descripción y códigos de respuesta
GET	/owners	Lista los propietarios. 200.
GET	/owners/{id}	Obtiene un propietario. 200 o 404.
POST	/owners	Registra un propietario. 201 (con la cabecera Location), 400 o 409.
PUT	/owners/{id}	Actualiza un propietario. 200, 400, 404 o 409.
DELETE	/owners/{id}	Elimina un propietario, siempre que no tenga mascotas. 204, 404, 409 o 503.

5.2 Pet-Service (puerto 8082)

Método	Ruta	Descripción y códigos de respuesta
GET	/pets	Lista las mascotas. 200.
GET	/pets/{id}	Obtiene una mascota. 200 o 404.
GET	/pets/owner/{ownerId}	Lista las mascotas de un propietario (la usa Owner-Service). 200.
GET	/pets/{id}/detalle	Mascota junto con los datos de su propietario (la usa Appointment-Service). 200, 404 o 503.
POST	/pets	Registra una mascota y valida que el propietario exista. 201, 400 o 503.
PUT	/pets/{id}	Actualiza una mascota (también cambia su estado). 200, 400, 404 o 503.
DELETE	/pets/{id}	Elimina una mascota. 204 o 404.

5.3 Appointment-Service (puerto 8083)

Método	Ruta	Descripción y códigos de respuesta
GET	/appointments	Lista las citas. 200.
GET	/appointments/{id}	Obtiene una cita. 200 o 404.
GET	/appointments/pet/{petId}	Lista las citas de una mascota. 200.
GET	/appointments/{id}/completa	Cita con su mascota y su propietario. 200, 404 o 503.
POST	/appointments	Programa una cita y valida que la mascota exista. 201, 400, 404, 409 o 503.

Método	Ruta	Descripción y códigos de respuesta
PUT	/appointments/{id}	Actualiza una cita (también cambia su estado). 200, 400, 404, 409 o 503.
DELETE	/appointments/{id}	Elimina una cita, siempre que no esté atendida. 204, 404 o 409.

5.4 Ejemplos de mensajes

Solicitud POST /owners

```
{
  "nombre": "Ana",
  "apellido": "Pérez",
  "documento": "1001",
  "telefono": "3001234567",
  "correo": "ana@mail.com",
  "direccion": "Calle 1"
}
```

Solicitud POST /pets

```
{
  "nombre": "Firulais",
  "especie": "Perro",
  "raza": "Labrador",
  "sexo": "MACHO",
  "fechaNacimiento": "2020-05-01",
  "peso": 25.5,
  "color": "Negro",
  "ownerId": 1
}
```

Solicitud POST /appointments

```
{
  "petId": 1,
  "fecha": "2030-01-15",
  "hora": "10:30",
  "motivoConsulta": "Vacunación",
  "observaciones": "Primera dosis"
}
```

Respuesta GET /appointments/1/completa (200)

```
{
  "cita": { "id": 1, "petId": 1, "fecha": "2030-01-15", "hora": "10:30:00",
            "motivoConsulta": "Vacunación", "observaciones": null, "estado": "PROGRAMADA"
},
  "mascota": { "id": 1, "nombre": "Firulais", "especie": "Perro", "raza": "Labrador",
               "sexo": "MACHO", "fechaNacimiento": "2020-05-01", "peso": 25.5,
               "color": "Negro", "ownerId": 1, "estado": "ACTIVO" },
  "propietario": { "id": 1, "nombre": "Ana", "apellido": "Pérez", "documento": "1001",
                   "telefono": "3001234567", "correo": "ana@mail.com",
```

```
                    "direccion": "Calle 1", "fechaRegistro": "2026-11-04T03:49:59.051986"
}
}
```


6. Reglas de negocio y códigos HTTP

Servicio	Regla	HTTP	Excepción
Owner	Documento o correo repetido	409	Conflicto
Owner	Teléfono obligatorio	400	ReglaNegocio
Owner	No se puede eliminar un propietario que tenga mascotas	409	Conflicto
Pet	La mascota debe tener un propietario que exista	400	ReglaNegocio
Pet	El nombre, la especie y la raza son obligatorios	400	ReglaNegocio
Pet	La fecha de nacimiento no puede ser futura	400	ReglaNegocio
Appointment	La mascota debe existir	404	NoEncontrado
Appointment	No se aceptan citas en fechas u horas que ya pasaron	400	ReglaNegocio
Appointment	No puede haber dos citas sin cancelar en la misma fecha y hora	409	Conflicto
Appointment	Una cita ATENDIDA no se puede modificar ni eliminar	409	Conflicto
Todos	El otro servicio está caído, responde con error 5xx o se agota el tiempo	503	ServicioNoDisponible
Todos	Campos que no cumplen las validaciones del DTO (@NotBlank, @Email, @Positive, etc.) o JSON mal formado	400	Validación de Spring

Las reglas viven en la capa de servicio y, cuando se puede, también como validaciones en los DTO. Así se pueden probar con pruebas unitarias sin necesidad de levantar la capa web.

7. Comunicación entre servicios

7.1 Mecanismo

Cuando nosotros vemos que un servicio necesita algo de otro, lo pide por HTTP con RestTemplate. Para eso cada uno cuenta con una clase cliente OwnerClient o PetClient que hace la llamada, pasa la respuesta a objetos de datos pensados para la integración y convierte los errores en excepciones propias. La conexión espera como máximo 2 segundos y la lectura 5 segundos, y esos tiempos se pueden ajustar con variables de entorno. Dentro de Docker Compose, los servicios se ubican entre sí por el nombre de su contenedor en la red veterinaria-net por ejemplo, http://pet-service:8082.7.2 Traducción de errores remotos que encontramemos etc

7.2 Traducción de errores remotos

Respuesta del servicio remoto	Excepción local	HTTP final
404 Not Found	NoEncontrado	404, o 400 al registrar una mascota (cuando el propietario no existe)
Conexión rechazada, timeout o 5xx	ServicioNoDisponible	503

7.3 Flujos principales

Operación	Llamadas realizadas
Registrar mascota (POST /pets)	Pet-Service → GET /owners/{ownerId} en Owner-Service. Si el propietario no existe: 400. Si Owner-Service no responde: 503.
Programar cita (POST /appointments)	Appointment-Service → GET /pets/{petId} en Pet-Service. Si la mascota no existe: 404. Si Pet-Service no responde: 503.
Cita completa (GET /appointments/{id}/completa)	Appointment-Service → GET /pets/{petId}/detalle en Pet-Service → GET /owners/{ownerId} en Owner-Service. Al final se devuelven la cita, la mascota y el propietario.
Eliminar propietario (DELETE /owners/{id})	Owner-Service → GET /pets/owner/{id} en Pet-Service. Si tiene mascotas: 409. Si Pet-Service no responde: 503 (no se borra nada).

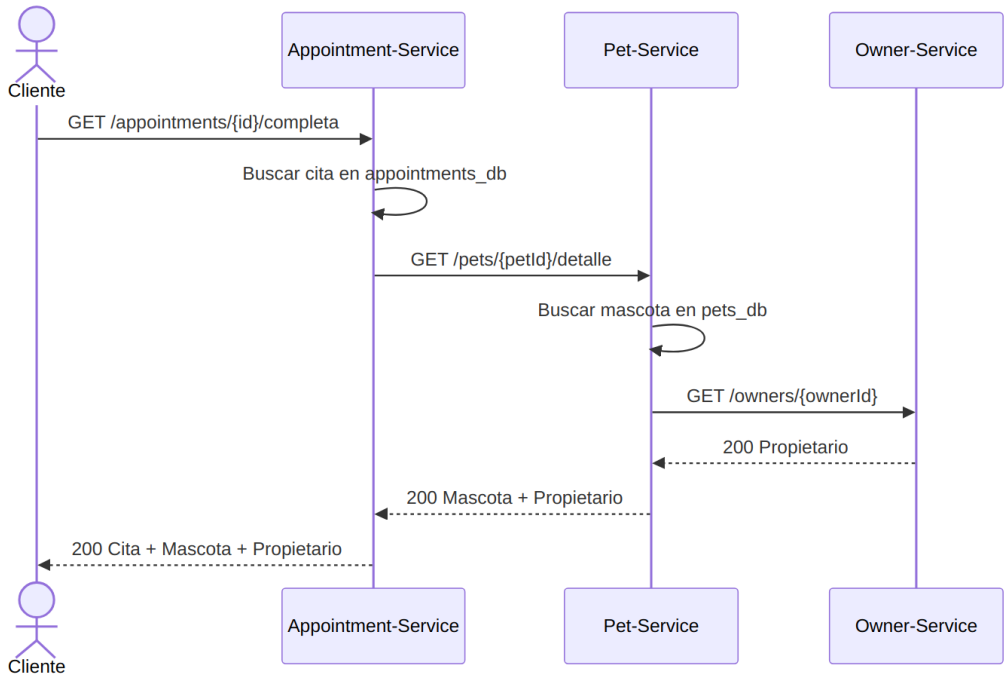


Figura 3. Secuencia de la consulta de cita completa.

8. Manejo de errores

Todos los errores salen con el mismo formato, gracias a un manejador global definido en Errores.java ejemplo este =

```
{
  "timestamp": "2026-11-04T03:50:40.353044872",
  "status": 404,
  "error": "Not Found",
  "mensaje": "La mascota con id 999 no existe",
  "ruta": "/appointments"
}
```

el manejador atiende los errores más comunes. cuando falla una regla de negocio responde con 404, 400, 409 o 503, según el caso. si los datos no pasan las validaciones, responde 400 y muestra qué campos fallaron y por qué. también responde 400 cuando el json viene mal escrito o cuando un parámetro llega con el tipo equivocado. si se rompe la integridad en la base de datos, devuelve 409. si la ruta no existe, devuelve 404, y si el método http no está permitido, devuelve 405. para cualquier error que no estaba previsto responde 500, y el detalle se guarda solo en el log del servidor.

9. Configuración

La configuración reside en src/main/resources/application.yml de cada servicio, Los valores sensibles a cada entorno se leen de variables de entorno con un valor por defecto para ejecución local como podermos ver en este caso.

Propiedad	Variable de entorno	Valor por defecto
spring.datasource.url	DB_URL	jdbc:mysql://localhost:3306/<base>
spring.datasource.username	DB_USER	root
spring.datasource.password	DB_PASSWORD	root
pet-service.url (Owner y Appointment)	PET_SERVICE_URL	http://localhost:8082
owner-service.url (Pet)	OWNER_SERVICE_URL	http://localhost:8081
clients.connect-timeout-ms	CLIENT_CONNECT_TIMEOUT_MS	2000
clients.read-timeout-ms	CLIENT_READ_TIMEOUT_MS	5000
server.port	(fijo en el archivo)	8081, 8082 u 8083
spring.jpa.hibernate.ddl-auto	(fijo en el archivo)	update

10. Despliegue con Docker

10.1 Imágenes

Cada servicio tiene un Dockerfile de varias etapas. En la primera se usa la imagen maven:3.9-eclipse-temurin-21 para descargar las dependencias y compilar el JAR (sin correr las pruebas). En la segunda se usa eclipse-temurin:21-jre, que es más liviana, solo para ejecutar el JAR.

```
FROM maven:3.9-eclipse-temurin-21 AS build
WORKDIR /app
COPY pom.xml .
RUN mvn -B -q dependency:go-offline
COPY src ./src
RUN mvn -B -q package -DskipTests

FROM eclipse-temurin:21-jre
WORKDIR /app
COPY --from=build /app/target/*.jar app.jar
EXPOSE 8081
ENTRYPOINT ["java", "-jar", "app.jar"]
```

10.2 Orquestación

El archivo docker-compose.yml define seis servicios dentro de la red veterinaria-net:

Contenedor	Puertos (equipo:contenedor)	Detalles
mysql-owners	3307:3306	mysql:8.4, base owners_db, volumen owners-data.
mysql-pets	3308:3306	mysql:8.4, base pets_db, volumen pets-data.
mysql-appointments	3309:3306	mysql:8.4, base appointments_db, volumen appointments-data.
owner-service	8081:8081	Espera a que mysql-owners esté saludable. PET_SERVICE_URL=http://pet-service:8082.
pet-service	8082:8082	Espera a que mysql-pets esté saludable. OWNER_SERVICE_URL=http://owner-service:8081.
appointment-service	8083:8083	Espera a que mysql-appointments esté saludable. PET_SERVICE_URL=http://pet-service:8082.

11. Estrategia de pruebas

las bases de datos tienen una revisión de salud, un healthcheck con mysqladmin ping, y configuramos los microservicios para que solo arranquen cuando su base ya está lista, usando depends_on con condition: service_healthy. si alguno falla, se reinicia solo gracias a la política on-failure. los datos se guardan en volúmenes con nombre, así que no se pierden al apagar los contenedores, y solo se borran si corres docker compose down -v. la contraseña de root la toma de la variable my sql , y si no le pones nada, usa root por defecto.

11.1 Tipos de prueba

Tipo	Descripción
Unitarias	Una clase por servicio (…ServiceTest) con JUnit 5 y Mockito. Simulan el repositorio y el cliente REST para probar únicamente las reglas de negocio.

Tipo	Descripción
Integración	Una clase por servicio (…ApiTest) con @SpringBootTest y MockMvc. Levantan el contexto completo con una base H2 en memoria (configurada con propiedades dentro de la misma anotación) y usan @MockBean para simular la llamada al otro servicio.

11.2 Casos del enunciado

Caso	Descripción	Resultado	Prueba de integración
1	Registrar propietario	201	OwnerApiTest.postOwners_devuelve201
2	Registrar mascota asociada	201	PetApiTest.postPets_conPropietarioValido_devuelve201
3	Programar cita	201	AppointmentApiTest.postAppointments_conMascotaValida_devuelve201
4	Cita para una mascota que no existe	404	AppointmentApiTest.postAppointments_mascotaInexistente_devuelve404
5	Mascota para un propietario que no existe	400	PetApiTest.postPets_conPropietarioInexistente_devuelve400

11.3 Resultados

Resultado de correr mvn test el 4 de noviembre de 2026 (Java 25.0.5, Spring Boot 3.3.5):

Servicio	Unitarias	Integración	Total	Resultado
Owner-Service	10	8	18	BUILD SUCCESS
Pet-Service	8	7	15	BUILD SUCCESS
Appointment-Service	9	7	16	BUILD SUCCESS
Total	27	22	49	0 fallos, 0 errores

Además, revisamos a mano con curl los cinco casos y las reglas de integridad contra los servicios desplegados con Docker Compose.

11.4 Colección de Postman

La carpeta postman trae la colección Clinica_Veterinaria.postman_collection.json, que automatiza la verificación de toda la API. Se ejecuta en orden: crea un propietario una mascota y una cita, guarda sus identificadores en las variables owner_id, pet_id y appointment_id, revisa el código de estado de cada petición y al final borra los datos que creó. Las direcciones base son variables de la colección (owner_url, pet_url y appointment_url) y por defecto apuntan a localhost. Se corre con Newman, la herramienta de línea de comandos de Postman por lo cual nosotros al hacerlo nos dio esto como resultado

```
npx newman run postman/Clinica_Veterinaria.postman_collection.json
```

Resultado que nos dio de la ejecución del 4 de noviembre de 2026 contra los tres servicios en Docker Compose, con las bases de datos vacías de esta forma:

Carpeta de la colección	Peticiones	Comprobaciones	Fallos
1. Owner-Service	6	6	0
2. Pet-Service	8	8	0
3. Appointment-Service	10	10	0
4. Reglas de eliminación	4	4	0
Total	28	28	0

La ejecución completa nos tardó 4,7 segundos con nuestros compañeros, con un tiempo medio de respuesta de 145 ms mínimo 10 ms y máximo 1258 ms. La primera petición a cada servicio es la más lenta porque todavía se está inicializando.

la colección deja una cita en estado atendida que ocupa el horario del 15 de enero de 2030 a las 10:30, así que no se puede correr dos veces seguidas sobre los mismos datos. Para repetirla hay que reiniciar las bases con docker,
con un jdk más nuevo que Java 21 por ejemplo Java 25, la versión de byte buddy que trae. Esto solo afecta a las pruebas que nosotros mismos hagamos

12. Decisiones de diseño

1. Pues sería Una base de datos por servicio y referencias lógicas entre ellos, para que los servicios no dependan unos de otros como tal.

2. Los dto son records de Java: no se pueden modificar, tienen las validaciones junto a cada campo y evitan exponer las entidades.
3. Los enumerados se guardan como texto y en mayúsculas por ejemplo en tratamiento
4. Una cita cancelada libera su horario y no cuenta al revisar citas duplicadas.
5. Toda cita nueva empieza en estado programada; el estado solo se cambia con PUT.
6. Si Pet-Service no responde, Owner-Service no deja borrar al propietario y devuelve 503: preferimos cuidar la integridad de los datos antes que la disponibilidad.
7. Pet-Service es quien arma el detalle de la mascota con su propietario, así Appointment-Service solo necesita conocer a Pet-Service y se mantiene la cadena .
8. Juntamos el código por filo para tener menos archivos y que el proyecto sea más fácil de leer.

13. Limitaciones y mejoras futuras

Limitación actual	Mejora propuesta
Las APIs no tienen autenticación ni autorización.	Agregar Spring Security con JWT y una puerta de enlace (API Gateway).
Las direcciones de los servicios se configuran a mano.	Usar descubrimiento de servicios (por ejemplo Eureka o el DNS de un orquestador).
Si una llamada a otro servicio falla, no se vuelve a intentar.	Agregar reintentos y un cortacircuitos (circuit breaker) con Resilience4j.
El esquema se crea con ddl-auto=update.	Manejar las migraciones con Flyway o Liquibase.
La contraseña de la base de datos es la de root, con un valor por defecto.	Usar un usuario con permisos limitados y guardar los secretos fuera del repositorio.
Para borrar propietarios, Owner-Service depende de Pet-Service.	Bajar el acoplamiento con eventos o con un contador local de mascotas.
No se puede seguir el rastro de una petición entre servicios.	Agregar trazas distribuidas y métricas (Spring Boot Actuator y Micrometer).